Debugging

(Also available in WeScheme)

Students confront various kinds of bugs and errors, and develop strategies for fixing them!

Lesson Goals	Students will be able to: • Identify and explain the difference between syntax, contract, and logic errors • Use strategies to correct each kind of error
Student-Facing Lesson Goals	• I know different kinds of coding errors, and can tell which is which • I know how to read an error message, and use it to help fix an error
Prerequisites	• Simple Data Types • Contracts • Functions: Contracts, Examples & Definitions • Solving Word Problems with the Design Recipe
Key Points for the Facilitator	• Meta-cognition is especially important here. Make sure you give *lots* of space for students to discuss their bug-fixing strategies • Productive struggle is critical in this lesson, to build up students' confidence in their ability to fix their own mistakes
Materials	• Lesson Slides • Bug Hunting Starter File • Syntax Errors (Page 25) • Contract Errors (Page 26) • Logic Errors (Page 27) • Logic Errors 2 (Page 28) • Logic Errors 3 (Page 29) • What Kind of Error? (Page 30)

Glossary

contract error :: errors where the code makes sense, but uses a function with the wrong number or type of arguments

logic error :: errors in the way a programmer is thinking about a problem

syntax error :: errors where the computer cannot make sense of the code (e.g. - missing commas, missing parentheses, unclosed strings)

Syntax Errors *15 min*

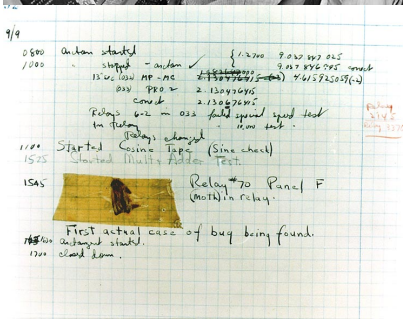
Overview

Students are introduced to *syntax errors* , the error messages they generate, and how to read those errors and fix them.

Launch

People often think that computers can't make mistakes, because they are "just machines". But who builds those machines? Who programs them? *Human beings* are always behind the scenes: writing code, designing chips, and even training Artificial Intelligences! **And humans make mistakes!**

In the 1940s, computers weren't made from tiny microprocessors. They were made from large wires, vacuum tubes, gears and dials. In 1946, the legendary programmer Grace Hopper (pictured) had a problem. She was certain that her code was right, but for some reason it wasn't running correctly.



She traced the wires and relays, and eventually found a *moth* that had gotten stuck and electrocuted! By disrupting the parts of the computer, it was causing the program to run incorrectly, and produce unexpected results! She wrote her findings into her notes, and even included the body of the moth. **This moth is the origin of the term "bug" in programming.**

Knowing how to find and fix bugs is a valuable skill for any programmer, and the kind of logical thinking you gain by fixing errors can help you find mistakes *outside* of a program as well!

When you Run a program, you expect the computer to do three things:

1. Read the code
2. Run the code
3. Give you the result you expected

In this lesson, we'll be talking about different kinds of mistakes, which can happen in each of those three phases:

1. ***Syntax Errors*** - Invalid code that the computer cannot even *read* , resulting in an error message. These are often typos, missing characters like a semicolon or parenthesis, or missing keywords like **`fun`** or **`end`** .
2. ***Contract Errors*** - Code that produces a value that breaks a function's contract, preventing the computer from *running* the program any further. For example, `circle("hello", "solid", "red"))` has no syntax errors. However, the contract for circle is `circle :: Number, String, String -> Image` so the the `circle` function is expecting a Number for it's first argument and "hello" is a String! Like syntax errors, contract errors will also result in an error message.
3. ***Logic Errors*** - Logic errors are often the hardest ones to find, because there is no error message! When there's a logic error in the code, the computer reads and runs the code just fine…but the result isn't what the programmer expected!

Investigate

Here are a few simple examples of syntax errors - can you find the problem?

- **`"hello`**
- `1+2`

Syntax Errors prevent the computer from *reading* code

- In pairs or small groups, complete Syntax Errors. For each piece of code, be sure to discuss as a team before deciding on the mistake.

Synthesize

Have students share back their findings. Leave time for discussion! The key here is to have students reflect on how useful it is to *read the error messages* first, even the few errors that may not be as helpful as the others.

1. Which syntax errors were the easiest to find? The hardest?
2. Which error messages were the most helpful? Least helpful?
3. Which syntax errors do *you* make the most often?
4. What strategies could we use to avoid making them in the first place?
5. What strategies could we use to fix them faster?

Contract Errors *15 min*

Overview

Students are introduced to *contract errors* . They read the error messages they generate, and discuss strategies to fix them.

Launch

With no syntax errors, the computer can read our code and try to run it. But a running program can still have problems! Here are a few examples of programs that have perfect syntax, but will generate an error when we try to run them. Can you spot the problem?

```
1 + "Zari"
triangle("50", "solid", "blue")
"true" or false
```

Contract Errors prevent the computer from *running* code

A program might be running along just fine, but as soon as a function is given the wrong type of value, the program halts with an error!

Investigate

In pairs or small groups, complete Contract Errors. For each piece of code, be sure to discuss as a team before deciding on the mistake.

Synthesize

Have students share back their findings. Leave time for discussion! When facilitating this discussion, drive home the point that reading the error and consulting the Contracts page are critical strategies for fixing these bugs.

1. Which contract errors were the easiest to find? The hardest?
2. Which error messages were the most helpful? Least helpful?
3. Which contract errors do *you* make the most often?
4. What strategies could we use to avoid making them in the first place?
5. What strategies could we use to fix them faster?

Logic Errors *20 min*

Overview

Students are introduced to *logic errors* , which are quite different from the other two kinds of errors! Logic errors are mistakes in **thinking** rather than **coding**.

Key point: This is where good habits like writing thorough examples and good comments are really helpful!

Launch

Ho-ming wanted to write a function to produce green triangles, and she went straight to coding the definition:

```
fun gt(size): triangle(100, "solid", "green") end
```

She clicked "Run" and didn't get any syntax errors, so she was feeling really confident. When she typed `gt(100)` she got a solid green triangle of size 100, and she was thrilled! But when she tried to make triangles of *different* sizes, her heart broke: all of the triangles were of size 100!

Did she have a syntax error? Why or why not? Did she have a contract error? Why or why not?

Ho-ming's mistake was that the function `gt` always made triangles of size 100! It took in `size` as a variable, but then didn't use it all. The computer had no trouble reading her code, and she followed the contract for `triangle` . *As far as the computer is concerned, there's nothing wrong with her code!*

Logic Errors don't prevent code from running at all!

Ho-ming's computer had no trouble reading or running her code. The problem is that the code didn't work the way she expected. Another way to think of it is that the bug isn't in the code at all - *it's in the way she was thinking when she wrote it.*

Logic Errors occur in our brains, not on the computer!

The only way to prevent logic errors - or to fix them when they happen! - is to be disciplined about the way we program. By thinking through a problem in multiple ways, we are less likely to make a mistake. What are some other ways Ho-ming could have thought through this function?

- **She could have written the Contract**, which might have put more emphasis on the input.
- **She could have written a Purpose Statement**, which would have forced her to talk about what happens to that input.
- **She could have written Examples**, which would have helped her see how the input is used. And even if she made the exact same mistake, at least her examples would have generated a warning to alert her to the problem!

Investigate

The Design Recipe helps us avoid logic errors, by demanding that we think through a problem in multiple ways. Even if *one* of our steps is wrong, we can check our work by comparing it to the other steps.

- In pairs or small groups, complete Logic Errors, Logic Errors 2, and Logic Errors 3.
- In pairs or small groups, open the Bug Hunting Starter File, and see if you can fix all of the syntax errors in the file by completing What Kind of Error?

Synthesize

Have students share back their findings. Leave time for discussion! The main idea for this discussion is that Logic Errors happen in the programmer's mind, *not* in the code. The best way to help prevent them is to think things through completely, and there are many strategies to do this. The Design Recipe, for example, forces students to think through the same solution in multiple representation - and the computer checks that those representations match.

1. Did you find any logic errors that you've made in the past?

2. What can you do in your own programming, to minimize the chances of logic errors?